

CS588 MP2: Center-Based 3D Detection From LiDAR

Name: Adwaith Mootheshath

NetID: adwaith3

Task 0: BEV Rasterization + Visualization

For this task, I built a 2D birds eye view map of the 3D lidar data by first sorting all the points into a grid and calculating their max height and density.

```
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$ python scripts/00_read_and_viz_kitti_raw.py
sequence=2011_09_26_drive_0005
num_frames=154
selected_frames=5 [0, 40)

--- BEV diagnostics (frame 0000) ---
  shape: (4, 400, 352) (expected (4, 400, 352))
  occupancy: 10156/140800 cells (7.2%)
  max_height   : range=[0.228, 1.000] mean=0.432
  mean_height  : range=[0.228, 0.998] mean=0.412
  intensity    : range=[0.001, 0.990] mean=0.241
  density      : range=[0.166, 1.000] mean=0.374

frame=0000 points=123397 gt_boxes=1 classes=['Car']
frame=0008 points=123411 gt_boxes=2 classes=['Car']
frame=0016 points=123129 gt_boxes=2 classes=['Car']
frame=0024 points=122430 gt_boxes=2 classes=['Car']
frame=0032 points=122703 gt_boxes=2 classes=['Car']
saved_visualizations=outputs/plots/01_kitti_raw
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$
```

Fig 1: Output screenshot

Task 1: Target Encoding

For this task, I built a translator to turn real world car coordinates into digital heatmap with some soft Gaussian glows and precise sub pixel offset.

```

adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$ python scripts/01_debug_dataloader_and_targets.py

--- BEV diagnostics (frame 0000) ---
shape: (4, 400, 352) (expected (4, 400, 352))
occupancy: 10156/140800 cells (7.2%)
max_height    : range=[0.228, 1.000] mean=0.432
mean_height   : range=[0.228, 0.998] mean=0.412
intensity      : range=[0.001, 0.990] mean=0.241
density        : range=[0.166, 1.000] mean=0.374

--- Encode diagnostics (frame 0000) ---
heatmap shape: (1, 100, 88) (expected (1, 100, 88))
heatmap max:   1.0000 (expected 1.0 at object centers)
encoded objects: 1 (gt boxes: 1)

frame=0000 gt=1 decoded=1 mean_center_err=0.0000m
frame=0001 gt=1 decoded=1 mean_center_err=0.0000m
frame=0002 gt=1 decoded=1 mean_center_err=0.0000m
frame=0003 gt=1 decoded=1 mean_center_err=0.0000m
frame=0004 gt=1 decoded=1 mean_center_err=0.0000m
frame=0005 gt=2 decoded=2 mean_center_err=0.0000m
frame=0006 gt=2 decoded=2 mean_center_err=0.0000m
frame=0007 gt=2 decoded=2 mean_center_err=0.0000m

--- Summary ---
overall_mean_center_error=0.0000m
PASS: encode/decode round-trip is consistent (error < 0.5m)
saved_visualizations=outputs/plots/02_targets
Check the images: green=GT boxes, red=decoded boxes. They should overlap closely.
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$

```

Fig 2: Output screenshot

Task 2: Model + Loss + Training Loop

For this task, I built a small CNN that takes the BEV LiDAR input and predicts where objects are along with their size, height, and rotation. I also set up the loss functions and training loop, and made sure the model could overfit the minival split as a sanity check.

```

[2026-04-23 20:09:18 + 75.9s] val_metric_stage_start frames=10 elapsed_s=0.54 rotated_iou=True
[2026-04-23 20:09:18 + 75.9s] val epoch=50 total=0.7000 hm=0.1415 reg=0.1310 z=0.1273 dims=0.196
8 yaw=0.1034 mAP=0.9560 train_time_s=0.55 val_time_s=0.63
[2026-04-23 20:09:20 + 77.7s] run_dir=outputs/processed_train/task2_minival_debug
[2026-04-23 20:09:20 + 77.7s] best_val_mAP=0.989011
[2026-04-23 20:09:20 + 77.7s] test_mAP=0.989011
[2026-04-23 20:09:20 + 77.7s] train_wall_time_s=75.323
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$ █

```

Fig 3: Output screenshot

Task 3: Inference Decoding

For this task, I took the model's outputs and turned them into actual 3D boxes by picking the top detections and decoding their position, size, and rotation. I also visualized the results to make sure the boxes aligned with the LiDAR data, and the precision and recall were high which confirmed it.

```
1 score_thresh=0.300000
2 f1=0.994475
3 precision=1.000000
4 recall=0.989011
5 tp=90
6 fp=0
7 fn=1
8
```

Fig 4: Output screenshot

Task 4: Full Train/Val

For this task, I trained the model on the full train and validation split. I did not need to change the learning rate or batch size to get better results. Also the mAP is improving each epoch, and passed the mAP threshold of test mAP ≥ 0.4 and test mAP ≥ 0.6 .

Question: While reading your curves, ask not just “is the loss going down?” but “does the detector become more decisive on centers without becoming geometrically worse on box regression?” The per-term losses (heatmap, reg, height, dims, rot) help answer this.

Ans: I checked the total loss and also the individual losses like heatmap, reg, dims, and yaw. Since the heatmap loss kept decreasing it shows the model is getting better at detecting object centers, and the regression losses also went down at the same time. This means the box predictions are getting more accurate. Since both improved together and the mAP increased, it means the model is learning correctly without hurting box quality.

```
[2026-04-23 20:43:58 + 811.7s] run_dir=outputs/processed_train/task4_full_train
[2026-04-23 20:43:58 + 811.7s] best_val_mAP=0.514598
[2026-04-23 20:43:58 + 811.7s] test_mAP=0.658012
[2026-04-23 20:43:58 + 811.7s] train_wall_time_s=781.046
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$
```

Fig 5: task4_full_train Output screenshot

```

[2026-04-23 21:12:03 + 785.1s] val epoch=10 total=2.9662 hm=1.6144 reg=0.3696 z=0.1751 dims=0.3373 yaw=0.4698 mAP=0.4524 train_time_s=56.88 val_time_s=25.57
[2026-04-23 21:12:33 + 814.7s] run_dir=outputs/processed_train/task4_bonus_point
[2026-04-23 21:12:33 + 814.7s] best_val_mAP=0.514598
[2026-04-23 21:12:33 + 814.7s] test_mAP=0.658012
[2026-04-23 21:12:33 + 814.7s] train_wall_time_s=784.039
adwaith3@jupyter-adwaith3:~/CS588-SP26/mp2$

```

Fig 6: task4_bonus_point Output screenshot

Task 5: Evaluation + PR Curves

For this task, I ran the evaluation script on the best checkpoint for the test split and got an AP of about 0.68 at IoU 0.5.

The PR curve stays pretty high for most of the recall range and only drops near the end. This means the model is making accurate predictions across different thresholds. Therefore the detector is consistent and not tuned to one score threshold.

```

split=test
checkpoint=outputs/processed_train/task4_full_train/best_val.pt
score_thresh=0.01
topk=100
nms_iou=0.1
iou_threshold=0.5
loss_total=inf
loss_heatmap=inf
loss_reg=0.248733
loss_height=0.115176
loss_dims=0.226281
loss_rot=0.412116
mAP=0.681576
pr_curve=outputs/processed_eval/task4_full_train_test/test_pr_curve.png
class=Car AP=0.681576 num_gt=5444 num_det=92852

```

1. Held-out test metrics (AP at IoU 0.5).

Ans: 0.68 at IoU 0.5 On the held-out test split, the model achieved an AP of 0.6816 at IoU 0.5. This indicates strong detection performance.

2. PR curve at IoU 0.5 and a brief interpretation of its shape.

Ans: The PR curve maintains high precision at low to mid recall, which shows the model is confident on its top predictions. As recall increases, precision drops slowly. This shows that lower confidence detections introduce more false positives. Overall, the curve is a good balance between precision and recall.

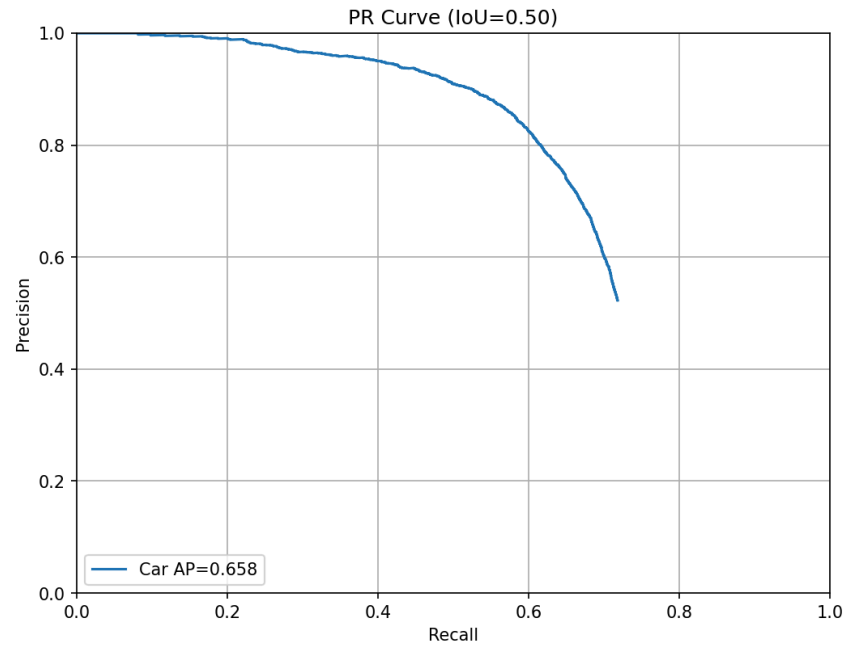


Fig 7: test_pr_curve screenshot

3. Your final train/val loss curves and per-epoch mAP.

Ans: The training loss decreases slowly across epochs, this confirms the model is learning effectively. The validation loss is more noisy but pretty much stable. This is expected due to generalization on unseen data. The validation mAP increased from around 0.2 in early epochs to about 0.5+ by the final epoch, showing steady improvement. This shows that the model improves its detection performance as training progresses.

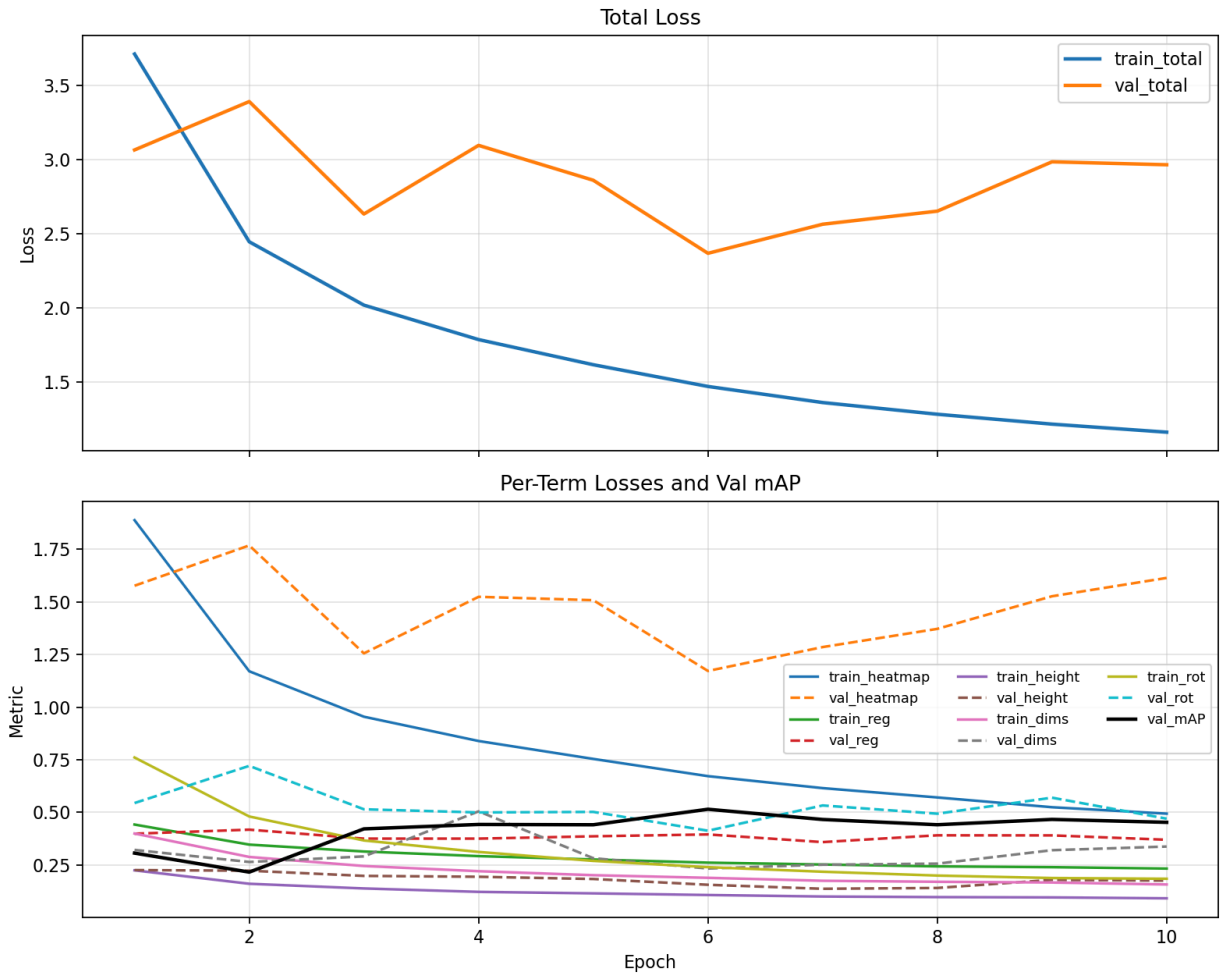


Fig 8: loss_curves screenshot

- Several qualitative visualization examples (use the best-F1 threshold from the line search).

Ans: The visualization examples some are shown below were generated using the best F1 threshold selected during inference. The predicted bounding boxes align well with the LiDAR point cloud and correctly capture object positions and orientations. This means that the model produces reasonable detections in both BEV and camera views.

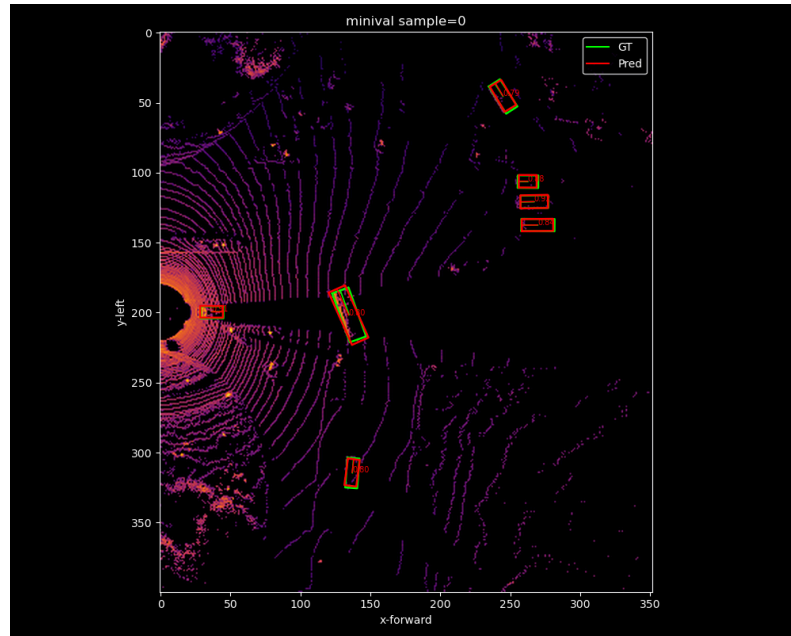


Fig 9: BEV example screenshot

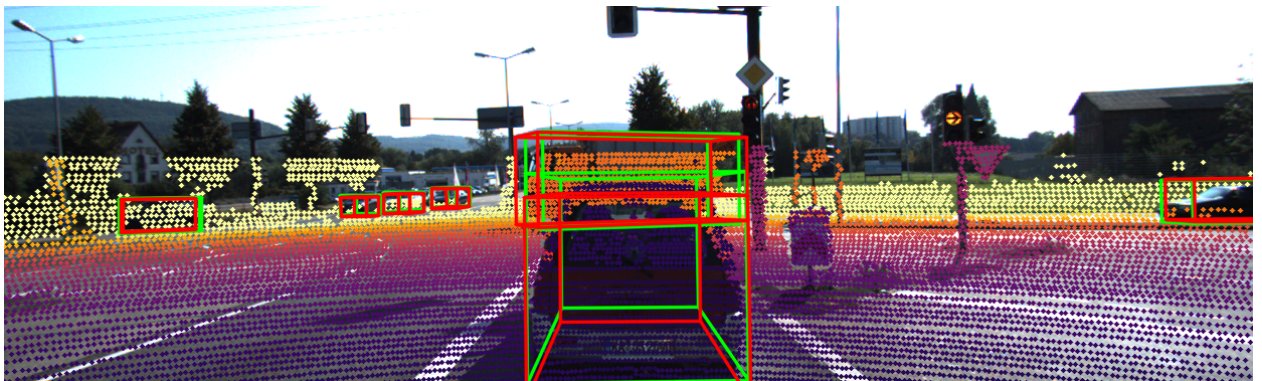


Fig 10: Cam example screenshot

5. Hyperparameters used for your final run.

Ans:

Hyperparameters used:

- epochs: 10
- batch size: 8
- validation batch size: 8
- learning rate: default
- weight decay: default
- IoU threshold: 0.5
- score threshold, top-k, and NMS were kept as default during evaluation

The model was trained for 10 epochs with a batch size of 8 for both training and validation. The evaluation IoU threshold was set to 0.5. Default hyperparameters provided in the script were used, and the model was trained on GPU.